UT1 · PROCESOS

Apuntes del alumno

Programación de Servicios y Procesos (PSP)
2.º DAM · Curso 2026–2027

PREGUNTA GUÍA

¿Qué ocurre desde que ejecutamos un programa hasta que el sistema operativo consigue que varias tareas avancen y podamos coordinar sus resultados?

Idea de la unidad

Comprender primero cómo funcionan los procesos; después utilizar Python para crearlos, observarlos, comunicarlos y repartir trabajo.

1. Qué vas a aprender

Un ordenador puede tener decenas o cientos de programas y servicios activos. Algunos calculan, otros esperan datos, otros leen archivos o atienden conexiones de red. El sistema operativo debe mantenerlos identificados, decidir qué ejecución puede avanzar y proporcionar mecanismos para que cooperen sin convertir toda la memoria en un espacio común.

En esta unidad construiremos ese modelo desde lo más básico y lo comprobaremos con programas Python pequeños. No se trata de memorizar funciones de una biblioteca, sino de poder explicar qué proceso ejecuta cada parte del código, qué datos pertenecen a cada proceso y cómo se recuperan los resultados.

AL TERMINAR LA UNIDAD DEBERÍAS PODER

Distinguir programa y proceso; interpretar PID y relación padre-hijo; explicar estados y planificación; diferenciar concurrencia y paralelismo; crear y supervisar procesos; comprender el aislamiento de memoria; comunicar resultados mediante IPC; y decidir cuándo repartir una carga entre varios procesos.

2. Programa y proceso no son lo mismo

Un programa es un conjunto de instrucciones y datos almacenados. Mientras no se ejecuta, es información pasiva. Un proceso es una instancia de ejecución gestionada por el sistema operativo (SO). Tiene identidad, recursos, estado y una actividad que evoluciona con el tiempo.

Al ejecutar dos veces el mismo archivo Python no obtenemos «el mismo proceso dos veces». Normalmente tendremos dos procesos distintos del intérprete de Python, cada uno con su propio ciclo de

vida y sus propios datos.

Descripción de instrucciones y datos	Instancia viva gestionada por el SO
Puede estar almacenado sin ejecutarse	Puede ejecutar, esperar, bloquearse o terminar

IMPORTANTE

Un proceso sigue existiendo aunque en ese instante no esté utilizando la CPU. Puede estar esperando una operación de entrada/salida, un temporizador o cualquier otra condición.

3. Identidad: PID, padre e hijo

Cada proceso recibe un PID (Process Identifier, identificador de proceso). El PID permite distinguir una instancia concreta de otras ejecuciones del mismo programa. El nombre del ejecutable no basta: pueden coexistir muchos procesos llamados python.exe.

Cuando un proceso crea otro, hablamos de proceso padre y proceso hijo. El PPID (Parent Process Identifier, identificador del proceso padre) permite representar esa relación cuando la plataforma la expone.

```
programa.py
├── proceso A PID 4312
├── proceso B PID 5076
└── proceso C PID 6210
```

NO MEMORICES UN PID

Los PID no son identificadores permanentes. Cuando un proceso termina, su número puede reutilizarse más adelante. Para diagnosticar una ejecución interesa combinar PID, nombre, instante de creación y, cuando sea posible, sus argumentos.

PARA SABER MÁS: CICLO DE VIDA, ZOMBIS Y HUÉRFANOS

En sistemas de tipo Unix pueden aparecer dos situaciones clásicas en la relación padre-hijo. Un proceso huérfano es un hijo cuyo padre termina antes que él; el sistema reasigna su relación de paternidad a un proceso encargado de recoger o supervisar descendientes, que puede ser init (PID 1) o un subreaper como systemd, según el sistema y la configuración. Un proceso zombi es un hijo que ya ha terminado pero cuyo estado de finalización todavía no ha sido recogido por su padre mediante una operación de espera. Ya no ejecuta código ni conserva su espacio de memoria de usuario, pero el kernel mantiene una pequeña entrada con información de terminación —incluido su PID— hasta que ese estado se recoge. En multiprocessing, join() realiza la espera necesaria para completar correctamente el ciclo de vida del hijo.

4. Recursos y espacio de direcciones

Un proceso necesita mucho más que código. El sistema mantiene información sobre su memoria,

archivos abiertos, directorio de trabajo, variables de entorno, credenciales, canales de comunicación y otros recursos.

Cada proceso trabaja normalmente con su propio espacio de direcciones virtual. Es una visión lógica de la memoria que el hardware y el sistema operativo traducen a memoria física. Dos procesos pueden utilizar el mismo valor de dirección virtual y, aun así, estar accediendo a datos diferentes.

PRECISIÓN ÚTIL

No es exacto decir que «cada proceso tiene una RAM distinta». Lo correcto es decir que dispone de un espacio de direcciones virtual propio y aislado por defecto. Puede existir memoria compartida, pero debe establecerse de forma explícita.

5. PCB: qué necesita recordar el sistema operativo

Para administrar una ejecución, el sistema operativo necesita conservar información sobre ella. Utilizamos el concepto PCB (Process Control Block, bloque de control del proceso) para representar ese conjunto de información.

- Identidad del proceso y relaciones con otros procesos.
- Estado actual: listo, ejecutando, bloqueado, etc.
- Información necesaria para la planificación: prioridad y otros datos.
- Contexto de ejecución: registros y punto por el que debe continuar.
- Referencias a memoria y recursos abiertos.
- Información de uso y contabilidad de recursos.

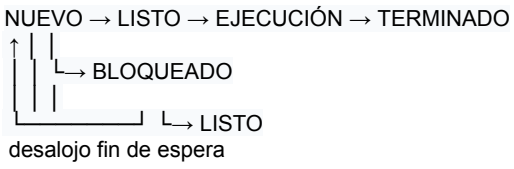
El PCB es un modelo conceptual. Los sistemas operativos reales utilizan estructuras internas distintas y no tienen por qué contener exactamente los mismos campos ni llamarlos igual.

IDEA CLAVE

El sistema puede detener temporalmente una ejecución y continuarla después porque conserva la información necesaria para saber quién es, qué recursos posee y por dónde debe seguir.

6. Estados de un proceso

Durante su vida, un proceso cambia de estado. El siguiente modelo es suficiente para razonar sobre lo que ocurre:



Los nombres exactos dependen del sistema operativo, pero el modelo permite distinguir situaciones fundamentales.

PRECISIÓN ÚTIL: TRANSICIONES IMPORTANTES

Ejecución → Listo: el planificador desaloja la ejecución, por ejemplo al agotarse su quantum o por una decisión de prioridad cuando otra tarea debe recibir el procesador en un sistema apropiativo. Ejecución

→ Bloqueado: la ejecución no puede continuar hasta que se cumpla una condición, por ejemplo al esperar una operación síncrona de E/S. Bloqueado → Listo: la condición termina, pero el proceso todavía debe obtener un nuevo turno de CPU.

Nuevo Se está creando y preparando.

Listo Puede ejecutar, pero espera que se le asigne CPU. Ejecución Está avanzando en el procesador. Bloqueado

No puede avanzar hasta que ocurra un evento o termine una espera.

Terminado	Ha finalizado su ejecución.

LISTO NO SIGNIFICA BLOQUEADO

Un proceso listo tiene todo lo necesario para ejecutar y solo espera turno de CPU. Un proceso bloqueado espera algo distinto del procesador: por ejemplo, datos de disco, red o la finalización de una operación.

Algunos modelos incluyen también estados suspendidos. La suspensión expresa que la ejecución ha sido detenida de forma más prolongada o por una decisión de gestión; no debemos identificarla automáticamente con «copiar toda su memoria al disco».

7. Concurrencia y paralelismo

Dos tareas son concurrentes cuando sus ejecuciones se solapan en el tiempo: ambas progresan durante un mismo intervalo. No es necesario que estén ejecutándose físicamente en el mismo instante.

Existe paralelismo cuando dos o más tareas ejecutan instrucciones realmente de forma simultánea, por ejemplo en distintos núcleos de una CPU (Central Processing Unit, unidad central de procesamiento).

CONCURRENCIA en 1 núcleo
A: ███ ███ ███
B: ██ ███

PARALELISMO en varios núcleos

A: ████████████
B: ████████████

PREGUNTA TÍPICA

¿Puede haber concurrencia con un solo núcleo? Sí. El sistema puede alternar rápidamente entre tareas. ¿Dos procesos garantizan paralelismo? No. Depende de que exista trabajo listo y de los recursos de ejecución disponibles.

8. CPU-bound e I/O-bound

No todas las tareas esperan por la misma razón. Esta distinción será importante al decidir cómo diseñar una aplicación concurrente.

🎬 CPU-bound: el tiempo está dominado por cálculo. La CPU es el recurso que limita el avance. 🎬 I/O-bound: gran parte del tiempo se dedica a esperar entrada/salida (E/S), como disco, red, terminal o dispositivos.

Añadir procesos puede ayudar en determinadas cargas de cálculo si hay varios núcleos y el trabajo compensa los costes de creación y comunicación. No existe la regla «más procesos = más velocidad».

9. Scheduler, dispatcher y quantum

El sistema operativo debe decidir qué ejecución lista recibe el procesador. Para estudiar esa decisión distinguimos dos ideas:

🎬 Scheduler (planificador): decide qué ejecución debería recibir el turno.
🎬 Dispatcher (despachador): hace efectiva la transferencia del control hacia la ejecución elegida. En algunos algoritmos aparece el quantum: una porción de tiempo tras la cual el sistema puede reconsiderar el turno. No existe un quantum universal y un proceso puede dejar la CPU antes si termina o se bloquea.

UNA PRECISIÓN QUE VEREMOS MEJOR EN UT2

Para construir el modelo podemos hablar de procesos, pero los sistemas operativos modernos suelen planificar entidades de ejecución más pequeñas, normalmente hilos. Un proceso puede contener varios hilos.

10. Cambio de contexto

Cuando el procesador deja una ejecución y continúa otra, el sistema debe conservar el estado necesario para reanudar la primera más adelante y recuperar el estado de la segunda. A esta operación la llamamos cambio de contexto.

Entre la información relevante están registros, contador de programa y puntero de pila. El PC (Program Counter, contador de programa) indica conceptualmente por qué instrucción debe continuar la ejecución; no es el número de línea del archivo Python.

PARA SABER MÁS: COSTE DEL CAMBIO DE PROCESO VS. HILO

Un cambio entre procesos puede implicar cambiar también el espacio de direcciones virtual, por lo que suele afectar a más estado de gestión de memoria que un cambio entre hilos del mismo proceso. Los procesadores y sistemas operativos modernos disponen de mecanismos como ASID (Address Space Identifier) o PCID (Process-Context Identifier) que permiten conservar determinadas traducciones en la TLB (Translation Lookaside Buffer, memoria caché de traducciones) y evitan tener que vaciarla por completo en cada cambio. El coste real depende de la arquitectura, del sistema operativo y de qué datos sigan siendo útiles en las distintas cachés; por tanto, no existe una penalización fija ni universal.

TIENE COSTE

Cambiar de contexto no realiza trabajo útil de nuestra aplicación. Además del coste directo de guardar y

recuperar estado, puede afectar a cachés y otros mecanismos del procesador. Por eso dividir el tiempo en porciones cada vez menores no hace que todo sea más rápido.

11. Modo usuario, modo kernel e interrupciones

Las aplicaciones normales se ejecutan en modo usuario, con privilegios limitados. Cuando necesitan una operación protegida —por ejemplo acceder a determinados recursos del sistema— utilizan interfaces controladas que permiten entrar en el kernel o núcleo del sistema operativo.

Una llamada al sistema solicita un servicio al kernel. Una interrupción permite que el procesador atienda un evento. Estos conceptos están relacionados con la gestión de procesos, pero no significan automáticamente que el sistema cambie a otro proceso.

NO CONFUNDAS

Cambiar de modo de privilegio, atender una interrupción y cambiar de contexto son operaciones diferentes. Una llamada al sistema puede terminar devolviendo el control a la misma ejecución.

12. Entrada/salida y procesos bloqueados

Supongamos que un proceso solicita datos de disco o de red. Mientras esos datos no estén disponibles, mantenerlo usando CPU sería inútil. El sistema puede bloquear esa ejecución y dejar avanzar otras tareas.

```
Proceso solicita E/S
↓
BLOQUEADO
↓ mientras tanto: otras tareas usan CPU
termina la E/S
↓
LISTO
↓
espera un nuevo turno de CPU
```

Cuando termina la E/S, el proceso normalmente pasa a listo. No tiene garantizado entrar inmediatamente en ejecución: todavía debe obtener turno.

13. Observar procesos reales

Antes de crearlos desde Python conviene observar procesos del propio sistema. En Windows podemos utilizar el Administrador de tareas y herramientas de línea de comandos; desde Python utilizaremos también la biblioteca psutil.

Al observar un proceso nos interesan, entre otros datos, PID, PPID, nombre, estado, consumo de CPU, memoria y número de hilos. Debemos recordar que el sistema cambia continuamente: un proceso puede terminar entre dos consultas.

OBSERVACIÓN ≠ VERDAD ABSOLUTA

Las herramientas muestran una vista del sistema y aplican sus propias unidades y formas de cálculo. Por ejemplo, distintos programas pueden normalizar de manera diferente el porcentaje de CPU.

14. Consultar procesos con psutil

psutil es una biblioteca de Python que permite consultar información de procesos y recursos del sistema. Crear un objeto psutil.Process no crea un proceso nuevo: representa uno que ya existe.

```
import psutil

# Obtiene información del proceso Python que ejecuta este script.
actual = psutil.Process()

# PID: identificador asignado por el sistema operativo.
print("PID:", actual.pid)
# Ruta del intérprete Python y directorio de trabajo actual.
print("Ejecutable:", actual.exe())
print("Carpeta:", actual.cwd())
# Estado que informa el sistema en el momento de la consulta.
print("Estado:", actual.status())
```

Al recorrer muchos procesos debemos asumir que algunos pueden desaparecer o que no tendremos permiso para consultar ciertos datos. Es mejor tratar esos casos concretos que ocultar cualquier error con un except genérico.

15. Ejecutar otros programas: subprocess.run

El módulo subprocess de la biblioteca estándar permite iniciar programas externos. Para los casos sencillos, subprocess.run() crea el proceso hijo y espera a que termine.

```
import subprocess
import sys

# sys.executable identifica el intérprete Python que usamos ahora.
# run() lanza el programa y espera a que termine.
resultado = subprocess.run(
 [sys.executable, "-c", "print(6 * 7)"],
 capture_output=True, # Captura stdout y stderr.
 text=True, # Devuelve el texto como str.
 check=True # Lanza excepción si el código de salida no es 0.
 )

# stdout es la salida impresa; returncode es el código de terminación.
print(resultado.stdout.strip())
```



```
print(resultado.returncode)
```

run() devuelve un objeto CompletedProcess. Su atributo returncode contiene el código de salida. Por convención, 0 suele indicar éxito, pero el significado real pertenece al contrato del programa ejecutado.

DIFERENCIA IMPORTANTE

Que el padre espere a su hijo no significa que todo el ordenador quede detenido. Otros procesos y otras ejecuciones pueden seguir avanzando.

16. Supervisar un proceso vivo: Popen

Cuando necesitamos crear el hijo y continuar ejecutando código en el padre, utilizamos subprocess.Popen. El objeto devuelto permite supervisar el proceso mientras sigue vivo.

```
import subprocess
import sys

# Popen inicia el hijo sin esperar a que termine.
p = subprocess.Popen([
 sys.executable, "-c",
 "import time; time.sleep(2)"
])

print("PID hijo:", p.pid)
# poll() devuelve None si el hijo sigue activo; si terminó, su código.
print("¿Ha terminado?:", p.poll())
# wait() bloquea al padre hasta la finalización del hijo.
codigo = p.wait()
print("Código:", codigo)
```

p.pid PID del hijo

p.poll() Consulta si ha terminado

p.wait() Espera su terminación

p.terminate()/p.kill()	Fuerzan o solicitan finalización según plataforma

CUIDADO CON POLL()

Mientras el hijo no haya terminado, poll() devuelve None. Cuando ha finalizado, devuelve su código de salida. No inviertas esta condición.

17. stdin, stdout y stderr

Los procesos suelen disponer de tres canales estándar: stdin (standard input, entrada estándar), stdout (standard output, salida estándar) y stderr (standard error, salida de error estándar). No tienen por qué corresponder físicamente a teclado y pantalla: pueden redirigirse a archivos o conectarse mediante tuberías.

subprocess.PIPE permite conectar esos canales con el proceso padre. Esto permite enviar datos a un programa y capturar por separado su salida normal y sus diagnósticos.

CUIDADO CON WAIT() Y LAS TUBERÍAS: DEADLOCK REAL

Si rediriges stdout o stderr con subprocess.PIPE y el hijo produce más datos de los que caben en la tubería, el hijo puede quedar bloqueado esperando que el padre lea. Si, al mismo tiempo, el padre está detenido en p.wait() esperando que el hijo termine, ninguno de los dos puede avanzar: se produce un

deadlock (interbloqueo). La capacidad de la tubería es limitada y depende del sistema; no debemos suponer un tamaño fijo. Para intercambios acotados en los que capturamos stdout/stderr, Popen.communicate() es el patrón recomendado: envía la entrada prevista, consume las salidas y espera la terminación. Para flujos muy grandes o continuos puede ser necesario diseñar lectura progresiva o redirigir la salida a un archivo.

SALIDA Y CÓDIGO DE RETORNO SON COSAS DISTINTAS

stdout transporta datos producidos por el programa. returncode describe cómo terminó. Un programa puede escribir en stderr y terminar correctamente, o producir stdout y terminar con un error.

18. Crear procesos Python con multiprocessing

subprocess es apropiado cuando queremos ejecutar otro programa. Cuando queremos ejecutar una función Python en otro proceso, podemos utilizar multiprocessing.Process.

```
import multiprocessing as mp
import os

# Esta función se ejecutará en un proceso hijo.
def trabajo(nombre):
 print(nombre, "PID", os.getpid())


def main():
 # spawn inicia un nuevo intérprete Python para el hijo.
 ctx = mp.get_context("spawn")
 # Construir Process no inicia todavía el proceso.
 p = ctx.Process(target=trabajo, args=("A",))
 p.start() # Arranca el hijo.
 p.join() # Espera su finalización; no recupera un return.
 print("Salida:", p.exitcode) # Código de salida del proceso.


# Impide volver a crear hijos al importar el módulo con spawn.
if __name__ == "__main__":
 main()
```

- Crear el objeto Process no inicia todavía el hijo.
- start() inicia el proceso.
- join() espera a que termine.
- is_alive() permite consultar si sigue vivo.
- exitcode informa de cómo terminó.
- join() no devuelve el valor retornado por la función.

ERROR MUY FRECUENTE

target=trabajo pasa la función al proceso. target=trabajo() ejecutaría la función inmediatamente en el padre y pasaría su resultado.

19. Por qué usamos if __name__ == "__main__"

Con el método de arranque spawn se inicia un intérprete nuevo para preparar el proceso hijo. El módulo principal debe poder importarse sin volver a crear hijos de forma accidental.

```
if __name__ == "__main__":
 main()
```

Esta protección no «activa el paralelismo». Evita que el código de creación de procesos se ejecute de nuevo cuando el módulo se importa durante el arranque del hijo. Para nuestros ejemplos, las funciones que ejecutarán los procesos se definirán a nivel de módulo.

20. Los procesos tienen memoria aislada

Una de las ideas más importantes de la unidad es que los procesos no comparten sus variables

normales. Pasar una lista como argumento a un proceso no significa que ambos procesos estén trabajando sobre la misma lista de memoria.

```
import multiprocessing as mp


def modifica(datos):
 # Modifica la copia de la lista que tiene el proceso hijo.
 datos.append(99)
 print("Hijo:", datos)


def main():
 datos = [1, 2]
 ctx = mp.get_context("spawn")
 # Con spawn, los argumentos se serializan para enviarlos al hijo.
 p = ctx.Process(target=modifica, args=(datos,))
 p.start()
 p.join()
 # La lista del padre permanece independiente de la del hijo.
 print("Padre:", datos)


if __name__ == "__main__":
 main()
```

PREDICE ANTES DE EJECUTAR

El hijo mostrará [1, 2, 99], pero el padre seguirá mostrando [1, 2]. Esperar con join() no copia las variables del hijo de vuelta al padre.

Declarar la variable como global tampoco la convierte en memoria compartida entre procesos. Si necesitamos recuperar un resultado, debemos establecer una comunicación explícita.

21. IPC: comunicación entre procesos

IPC (Inter-Process Communication, comunicación entre procesos) es el conjunto de mecanismos que permiten intercambiar información entre ejecuciones aisladas. La comunicación no elimina el aislamiento: crea una vía controlada entre procesos.

Existen muchos mecanismos de IPC: tuberías, colas, memoria compartida, archivos, sockets y otros. En esta unidad utilizaremos principalmente Pipe y Queue de multiprocessing. Los sockets se estudiarán más adelante en la unidad de redes.

REGLA OPERATIVA: SERIALIZACIÓN CON PICKLE

En nuestras prácticas, basadas en spawn, multiprocessing.Pipe, multiprocessing.Queue y ProcessPoolExecutor, los argumentos, funciones de trabajo y mensajes que deban atravesar la frontera

entre procesos tienen que poder serializarse según el mecanismo que utiliza multiprocessing, normalmente pickle. El objeto se representa para poder transferirlo y se reconstruye en el proceso destino: no se comparte la referencia original. Las funciones lambda o funciones locales suelen fallar con spawn porque no pueden localizarse/importarse como exige este modelo, y muchos objetos ligados a recursos externos necesitan un tratamiento específico. El tipo de excepción concreto depende del objeto y del punto en que falle la serialización, por lo que no debe esperarse siempre PicklingError.

22. Un resultado mediante Pipe

Pipe crea dos extremos de comunicación. En una tubería unidireccional, uno envía y el otro recibe. El mensaje debe contener toda la información que el receptor necesite para interpretarlo.

```
import multiprocessing as mp


def calcula(envio, numero):
 # El hijo envía una tupla: dato original y su cuadrado.
 envio.send((numero, numero * numero))
 envio.close()


def main():
 ctx = mp.get_context("spawn")

 # Tubería de una sola dirección:
 # primer extremo para recibir; segundo para enviar.
 recepcion, envio = ctx.Pipe(duplex=False)

 # El hijo recibe el extremo de envío y el número.
 p = ctx.Process(target=calcula, args=(envio, 7))
 p.start()

 # El padre no necesita su copia del extremo de envío.
 envio.close()

 # Espera el mensaje del hijo y desempaqueta sus dos valores.
 numero, resultado = recepcion.recv()
 recepcion.close()

 # Espera la finalización del hijo.
 p.join()

 print("Número:", numero)
 print("Cuadrado:", resultado)
 print("Código de salida:", p.exitcode)


if __name__ == "__main__":
 main()
```

MENSAJE, NO VARIABLE COMPARTIDA

El receptor obtiene un objeto reconstruido a partir de los datos enviados. No recibe una referencia remota a la variable original del otro proceso.

23. Varios resultados mediante Queue

Una Queue permite que varios procesos depositen mensajes y que otro proceso los recoja. Es

especialmente útil cuando hay varios productores de resultados.

```
import multiprocessing as mp


def calcula(cola, numero):
 # Cada hijo publica un resultado identificado por su número.
 cola.put((numero, numero * numero))


def main():
 ctx = mp.get_context("spawn")
 cola = ctx.Queue() # Cola compartida mediante comunicación entre procesos.
 numeros = (2, 3, 4)
 hijos = [ctx.Process(target=calcula, args=(cola, n)) for n in numeros]

 for p in hijos:
 p.start() # Inicia los tres procesos.

 # Sabemos que esperamos un mensaje por hijo; su orden puede variar.
 recibidos = [cola.get(timeout=10) for _ in hijos]

 for p in hijos:
 p.join() # Confirma que todos han terminado.

 print("Resultados:", recibidos)
 cola.close() # Cierra la cola en el padre.
 cola.join_thread() # Espera el cierre de su hilo de transferencia.


if __name__ == "__main__":
 main()
```

El orden de llegada puede cambiar. Por eso cada resultado debería incluir una identidad de tarea. Si llegan primero los datos de la tarea 5, el receptor debe poder saber que pertenecen a esa tarea.

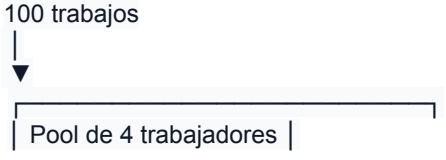
NO USES EMPTY() PARA DECIDIR QUE TODO HA TERMINADO

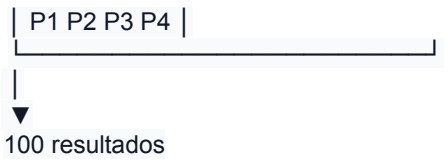
Que una cola esté vacía ahora solo significa que no hay un mensaje disponible en ese instante. Un productor podría enviar otro inmediatamente después. El final debe formar parte del protocolo: número de mensajes esperados, centinelas u otro acuerdo.

24. Trabajos, workers y pools

Un worker o trabajador es una ejecución dedicada a realizar tareas. No es un tipo especial de proceso del sistema operativo. Un pool mantiene un conjunto limitado de trabajadores para reutilizarlos.

Es importante distinguir trabajo y trabajador. Podemos tener 100 tareas y solo 4 procesos trabajadores. Cada trabajador irá realizando varias tareas.





DISEÑAR EL REPARTO IMPORTA

Crear un proceso para cada operación diminuta suele ser ineficiente. Hay costes de creación, planificación, serialización y comunicación. Normalmente agrupamos trabajo y reutilizamos un número limitado de procesos.

25. ProcessPoolExecutor

ProcessPoolExecutor ofrece una abstracción de alto nivel para gestionar un conjunto de procesos trabajadores. Nos permite concentrarnos en las tareas y resultados sin programar manualmente todo el ciclo del pool.

```
from concurrent.futures import ProcessPoolExecutor
import multiprocessing as mp


def cuadrado(n):
 # El worker devuelve un par: número y resultado.
 return n, n * n


def main():
 # Elegimos explícitamente spawn para el conjunto de trabajadores.
 ctx = mp.get_context("spawn")

 # El bloque with libera los recursos del pool al salir.
 with ProcessPoolExecutor(
 max_workers=2, # Hasta dos procesos trabajadores.
 mp_context=ctx
 ) as pool:
 # map reparte las tareas y entrega los resultados en orden de entrada.
 resultados = list(pool.map(cuadrado, range(6)))

 print(resultados)


if __name__ == "__main__":
 main()
```

max_workers=2 significa que el pool puede utilizar hasta dos trabajadores, no que vaya a crear un proceso diferente para cada tarea. map entrega los resultados en el orden de las entradas, aunque el orden real de ejecución pueda ser diferente.

submit devuelve un Future, un objeto que representa un resultado que puede no estar disponible todavía. result() permite obtenerlo cuando esté listo o recibir la excepción producida por la tarea.

26. ¿Varios procesos hacen el programa más rápido?

No necesariamente. Para decidirlo debemos medir. Una comparación correcta exige que la versión secuencial y la versión repartida realicen el mismo trabajo y produzcan el mismo resultado.

- Crear procesos tiene coste.
- Enviar y reconstruir datos tiene coste.
- Coordinar resultados tiene coste.
- Una tarea demasiado pequeña puede tardar más al repartirla.
- Una carga CPU-bound suficientemente grande puede beneficiarse de varios núcleos.
- Una parte secuencial importante limita la aceleración total.

RESULTADO VÁLIDO

Si la versión secuencial tarda 2 s y la versión con procesos tarda 3 s, eso no demuestra por sí solo que el código esté mal. Puede significar que los costes de repartir el trabajo son mayores que la ganancia obtenida.

27. Errores de diseño que debes reconocer

Dormir 1 s para «esperar al hijo»	El tiempo no garantiza que haya terminado; usa join()/wait().
Modificar una lista global desde el hijo	La memoria ordinaria está aislada entre procesos.
Crear un proceso por cada operación mínima	El coste de coordinación puede dominar.
Suponer que los mensajes llegan en orden	La planificación puede variar; identifica cada tarea.
Usar Queue.empty() como señal de fin	Vacía ahora no significa que no llegarán más mensajes.
Matar todos los python.exe	Puede destruir procesos ajenos; controla solo los hijos propios.

28. Práctica integradora M1

Construiremos un procesador de lotes numéricos. El coordinador dividirá los números del 1 al 50 en lotes, encargará a procesos trabajadores la suma de cuadrados de cada lote y reunirá los resultados.

También existirá una ejecución secuencial equivalente. Ambas deben producir el mismo total: 42925.

CRITERIOS TÉCNICOS DE LA SOLUCIÓN

La solución utilizará el contexto spawn y el gestor with ProcessPoolExecutor(...) para cerrar correctamente el pool. Los resultados volverán de forma explícita al coordinador; no se utilizarán variables globales como mecanismo de memoria compartida.

- Cada resultado identificará el lote y el PID del trabajador.
- El coordinador comprobará que no falta ni se repite ningún lote.
- El orden de llegada no afectará al resultado.
- Se utilizará ProcessPoolExecutor con contexto spawn.
- Se compararán tiempos sin afirmar que más procesos siempre son mejores.

QUÉ SE EVALÚA REALMENTE

No basta con que el programa produzca 42925. Debes poder explicar qué proceso ejecuta cada función, por qué los resultados vuelven al coordinador, qué ocurriría si un lote falla y por qué el número de workers elegido tiene sentido.

29. Qué debes saber explicar sin mirar código

1. Diferencia entre programa y proceso.
2. Qué representan PID y PPID y por qué el PID no es permanente.
3. Qué significa que un proceso tenga espacio de direcciones virtual propio.
4. Para qué sirve el modelo PCB.
5. Diferencia entre listo y bloqueado.
6. Diferencia entre concurrencia y paralelismo.
7. Qué hacen scheduler, dispatcher y cambio de contexto.
8. Por qué una operación de E/S puede bloquear una ejecución.
9. Diferencia entre subprocess.run() y subprocess.Popen().
10. Qué hacen start(), join(), is_alive() y exitcode en multiprocessing.Process.
11. Por qué if __name__ == "__main__" es importante con spawn.
12. Por qué una variable modificada en un hijo no cambia automáticamente en el padre. 13. Qué significa IPC y cuándo usar Pipe o Queue.
14. Diferencia entre trabajo, worker y pool.
15. Por qué ProcessPoolExecutor no garantiza que más procesos reduzcan el tiempo.

30. Comprueba que lo has entendido

16. Ejecutas dos veces el mismo archivo Python. ¿Tienes un programa o dos? ¿Cuántos procesos esperas normalmente?
17. Un proceso está listo y otro bloqueado. ¿Cuál de los dos podría utilizar CPU inmediatamente si recibe turno?
18. ¿Puede existir concurrencia en un equipo con un único núcleo? ¿Y paralelismo real de dos cálculos al mismo tiempo?
19. p.poll() devuelve None. ¿Qué puedes afirmar sobre el hijo en ese instante?
20. ¿Por qué p.join() no devuelve el resultado de la función ejecutada por el hijo?

21. Un hijo añade 99 a una lista recibida del padre. Después de join(), ¿por qué el padre puede seguir viendo la lista original?
22. Una Queue está vacía. ¿Puedes concluir que ningún proceso enviará más resultados? 23. Hay 20 trabajos y max_workers=2. ¿Cuántos PID distintos deberían aparecer obligatoriamente? 24. Una versión con cuatro procesos es más lenta que la secuencial. Enumera al menos tres causas posibles. 25. ¿Por qué es peligroso confundir stdout con el código de retorno?

31. Resumen de la unidad

MAPA MENTAL

PROGRAMA → PROCESO → IDENTIDAD Y RECURSOS → ESTADOS → PLANIFICACIÓN → OBSERVACIÓN → CREACIÓN → AISLAMIENTO → IPC → WORKERS/POOL → MEDICIÓN

Un proceso es una ejecución gestionada, no simplemente un archivo ni una tarea que esté usando CPU continuamente. El sistema operativo conserva la información necesaria para identificarlo, administrar sus recursos y detener o reanudar su ejecución.

La concurrencia permite que varias actividades progresen durante el mismo intervalo; el paralelismo exige ejecución simultánea. La planificación, los bloqueos de E/S y los cambios de contexto explican buena parte del comportamiento observable.

Desde Python podemos ejecutar programas externos con subprocess y crear procesos que ejecutan funciones con multiprocessing. Como los procesos tienen memoria aislada por defecto, los resultados deben volver mediante mecanismos explícitos de comunicación. Los pools permiten reutilizar trabajadores y repartir tareas, pero su utilidad debe comprobarse midiendo.

32. Vocabulario esencial

SO Sistema operativo.

CPU Central Processing Unit: unidad central de procesamiento. PID Process IDentifier: identificador de proceso.

PPID Parent Process IDentifier: identificador del proceso padre. PCB Process Control Block: modelo del bloque de información que el SO conserva para gestionar un proceso.

E/S (I/O) Entrada/salida: operaciones con archivos, red, terminales o dispositivos.

IPC Inter-Process Communication: comunicación entre procesos. Scheduler Planificador: decide qué ejecución debería recibir turno. Dispatcher Despachador: hace efectiva la transferencia de control. Quantum Porción de tiempo utilizada por determinadas políticas de planificación. Context switch Cambio de contexto entre ejecuciones.

Worker Trabajador que ejecuta tareas.

Pool Conjunto de trabajadores reutilizables.

Future	Objeto que representa un resultado que puede estar pendiente.